

SolarPulse Project - Comprehensive DAX & Key Metrics Documentation

This official documentation provides a detailed breakdown of the 37 production-level DAX measures implemented in the Power BI dashboard for the SolarPulse renewable energy project.

1. Technical Performance & Efficiency Metrics

Expected_Power_AC

Calculates the theoretical AC power output based on solar irradiation, total panel area, and panel efficiency.

```
Expected_Power_AC = VAR Total_Irr = SUM("Plant_1_Generation_Data"[IRRADIATION])  
VAR Area = 10000 -- Total plant area in square meters  
VAR Eff = 0.18 -- Panel efficiency (18%)  
RETURN Total_Irr * Area * Eff
```

Inverter Efficiency %

Calculates the efficiency of the inverters by dividing the converted AC power by the input DC power.

```
Inverter_Efficiency_% = VAR Total_AC = SUM("Plant_1_Generation_Data"[AC_POWER])  
VAR Total_DC = SUM("Plant_1_Generation_Data"[DC_POWER])  
RETURN DIVIDE(Total_AC * 10, Total_DC, 0)
```

Performance_Ratio

The ultimate engineering health check indicator. Dynamically adjusts the surface area depending on whether a single inverter or the whole plant is selected.

```
Performance_Ratio = VAR Total_AC = SUM("Plant_1_Generation_Data"[AC_POWER])  
VAR Total_Irr = SUM("Plant_1_Generation_Data"[IRRADIATION])  
VAR Correct_Area = IF(ISFILTERED("Plant_1_Generation_Data"[SOURCE_KEY]), 454.5, 10000)  
VAR Efficiency = 0.18  
RETURN IF(Total_Irr > 0, DIVIDE(Total_AC, (Total_Irr * Correct_Area * Efficiency)), BLANK())
```

Power_Loss

Measures the lost power in kW due to environmental or technical drops (Expected vs. Actual).

```
Power_Loss = [Expected_Power_AC] - SUM("Plant_1_Generation_Data"[AC_POWER])
```

Specific Yield (kWh/kWp)

Measures the plant's productivity normalized by its size.

Specific Yield (kWh/kWp) = $\text{DIVIDE}([\text{Actual Energy (kWh)}], [\text{Installed Capacity kWp}], 0)$

TargetEfficiency

Standard benchmark threshold for plant operational health.

TargetEfficiency = 0.75

2. Power & Energy Generation Metrics

Total AC Power (kWh)

Total AC Power (kWh) = SUM("Plant_1_Generation_Data"[AC_POWER])

Actual Power AC (kW)

Normalizes the raw sensor AC data to standard kilowatts.

Actual Power AC (kW) = DIVIDE(SUM("Plant_1_Generation_Data"[AC_POWER]), 100, 0)

Actual Energy (kWh)

Converts instant power readings to cumulative energy based on the 15-minute sensor intervals.

Actual Energy (kWh) = [Actual Power AC (kW)] * 0.25

Expected Energy (kWh)

Expected Energy (kWh) = [Expected_Power_AC] * 0.25

Daily Production (kWh)

Daily Production (kWh) = FORMAT(DIVIDE(SUMX(SUMMARIZE("Plant_1_Generation_Data", "Plant_1_Generation_Data"[DATE], "Plant_1_Generation_Data"[SOURCE_KEY], "DailyValue", MAX("Plant_1_Generation_Data"[DAILY_YIELD])), [DailyValue]), 100, 0), "#,##0.0")

Avg Daily Yield

Avg Daily Yield = AVERAGE("Plant_1_Generation_Data"[DAILY_YIELD])

Today Yield

Filters and aggregates the AC power exclusively generated starting from the latest recorded date timestamp.

Today Yield = VAR LastTimestamp = MAX("Plant_1_Generation_Data"[DATE_TIME]) VAR LastDateOnly = DATE(YEAR(LastTimestamp), MONTH(LastTimestamp), DAY(LastTimestamp)) RETURN CALCULATE(SUM("Plant_1_Generation_Data"[AC_POWER]), "Plant_1_Generation_Data"[DATE_TIME]>= LastDateOnly)

Lifetime Total Energy (MWh)

Lifetime Total Energy (MWh) = DIVIDE([Actual Energy (kWh)], 1000, 0)

Peak Power

Peak Power = MAXX(KEEPFILTERS(VALUE("Plant_1_Generation_Data"[DATE_TIME])),
[Total AC Power (kWh)])

Sum of Avg Expected Power AC

Sum of Avg Expected Power AC = SUMX(
VALUES("Plant_1_Generation_Data"[SOURCE_KEY]),
CALCULATE(AVERAGE("Plant_1_Generation_Data"[AC_POWER])))

3. Operational Time & Weather Sensors

Total Planned Time

Calculates total theoretical daylight operating hours based on positive irradiation sensor rows multiplied by 15-minute time intervals.

Total Planned Time = COUNTROWS(FILTER("Plant_1_Weather_Sensor_Data",
"Plant_1_Weather_Sensor_Data"[IRRADIATION] > 0)) * 0.25

Actual Operating Time (H)

Actual Operating Time (H) = COUNTROWS(FILTER("Plant_1_Generation_Data",
"Plant_1_Generation_Data"[AC_POWER] > 0)) * 0.25

Avg Irradiation (Daylight)

Avg Irradiation (Daylight) = AVERAGEX(FILTER("Plant_1_Weather_Sensor_Data",
"Plant_1_Weather_Sensor_Data"[IRRADIATION] > 0),
"Plant_1_Weather_Sensor_Data"[IRRADIATION])

Max Irradiation

Max Irradiation = MAX("Plant_1_Weather_Sensor_Data"[IRRADIATION])

Max Module Temp C

Max Module Temp C = MAX("Plant_1_Weather_Sensor_Data"[MODULE_TEMPERATURE])

4. Sustainability & Environmental Impact

Carbon Offset (tCO2)

Carbon Offset (tCO2) = [Lifetime Total Energy (MWh)] * 0.8

CO2 Saved (Tons)

CO2 Saved (Tons) = ([Total AC Power (kWh)] * 0.5) / 1000

Daily_CO2_Saved

Daily_CO2_Saved = VAR Daily_MWh = SUM("Plant_1_Generation_Data"[AC_POWER]) / 1000 RETURN Daily_MWh * 0.0007

Coal Avoided (Lbs)

Coal Avoided (Lbs) = [Total AC Power (kWh)] * 0.28

Equivalent Trees

Converts carbon offset into the equivalent number of fully grown trees required to absorb the same amount of CO2.

Equivalent Trees = DIVIDE([Carbon Offset (tCO2)] * 1000, 21.7, 0)

Equivalent Car Miles

Equivalent Car Miles = DIVIDE([Carbon Offset (tCO2)] * 1000, 0.4, 0)

Smartphones Charged

Smartphones Charged = [Total AC Power (kWh)] * 41.4

5. Python Machine Learning Integration

Fix_Predicted

Integrates and maps the external Python predictive machine learning algorithm results directly into Power BI visuals, syncing the values dynamically via a unified time-index mapping logic.

```
Fix_Predicted = CALCULATE( AVERAGE(Predictive_Model_Python[Predicted_AC_Power]),  
TREATAS(VALUEs(Query_SQL[TIME]), Predictive_Model_Python[TIME]) )
```